

Nao

A multi-agent AI framework in F# with structured orchestration, memory management, the ETCLOVG seven-layer harness architecture, pluggable tool execution, and Orleans-based distributed multi-tenant runtime.

Overview

Nao is a framework for building composable AI agents that can reason, collaborate, and persist state. It provides structured prompt engineering, tool invocation with content-type awareness and revert capabilities, multi-agent orchestration patterns, conversation history management, semantic memory, governance, observability, and verification — all running on Microsoft Orleans for scalable distributed multi-tenant execution.

The framework implements the **ETCLOVG** taxonomy from "Agent Harness Engineering: A Survey" — seven layers that govern every agent execution:

Layer	Concern	Key Types
E — Execution	Resource-bounded sandboxed execution	ExecutionContext, ResourceLimits, SandboxConfig
T — Tool Protocol	Structured tool discovery, middleware, verify/revert	IToolProtocol, ToolSchema, IToolMiddleware, ExecutionJournal
C — Context & Memory	Tiered memory, context compaction	ITieredMemory, ContextCompaction, MemoryTier
L — Lifecycle	State-machine lifecycle, pipeline stages	AgentLifecycle, LifecyclePipeline, RetryPolicy
O — Observability	Distributed tracing, metrics, resilience	ITracer, IMetricsCollector, CircuitBreaker
V — Verification	Readiness checks, execution traces, regression	IReadinessCheck, ExecutionTrace, IJudge
G — Governance	Permissions, constitution, audit, policies	PermissionModel, Constitution, PolicyEngine

Features

- **ETCLOVG Harness** — Seven-layer execution pipeline with resource bounds, governance, observability, and verification
- **Multi-Agent Orchestration** — Router, Pipeline, and AgentGroup patterns for composing agents
- **Extensible Orchestrator** — Abstract base class with virtual members (TryParseAction, BuildSystemPrompt) for custom behavior via inheritance and DI
- **Conversation Memory** — Sliding window, token-budget, summarization, and tiered memory strategies
- **Semantic Memory** — Embedding-based retrieval for long-term agent knowledge
- **Persistent State** — Orleans grain persistence for conversation history and memories across sessions

- **Structured Prompts** — Type-safe prompt engineering with roles, constraints, examples, and output formats
- **Tool Protocol** — MCP-inspired tool discovery with middleware, rate limiting, and schemas
- **Content Metadata** — Generic ContentMeta type lets tools/agents declare output types (JSON, PDF, images, etc.)
- **Tool Verify & Revert** — Tools can declare verify (check correctness) and revert (undo side-effects) capabilities
- **Execution Journal** — Immutable log of all tool executions; supports bulk revert of revertible operations
- **Pluggable Tool Execution** — Tools run as processes, HTTP calls, or custom executors (gRPC, MCP, etc.)
- **Governance** — Constitution rules, permission models, audit logging, and runtime policy enforcement
- **Observability** — Distributed tracing (OpenTelemetry-style), cost metrics, circuit breakers, retries
- **Verification** — Readiness gates, execution trace capture, LLM judges, regression detection
- **Evaluation** — Test case framework with multiple evaluators, LLM judges, and dataset-level reports
- **Multi-Provider Support** — Pluggable LLM backends (OpenAI, Anthropic, Ollama, vLLM, llama.cpp)
- **Workspace Loader** — JSON definitions and assembly plugin discovery for agents, tools, and evals
- **Multi-Workspace Runtime** — Multiple isolated workspaces within a single Orleans silo with dynamic hot-reload
- **Group Directory** — Organizational multi-tenancy: groups own sessions, members, and default workspaces
- **Desktop Assistant** — Avalonia.FuncUI chat app with an embedded ASP.NET Core + Orleans server: real-time execution-trace streaming, dark/light theme switching, and a localizable UI
- **F# First** — Immutable records, discriminated unions, and functional composition throughout

Project Structure

```

Nao.slnx
├── src/
│   ├── Nao.Core/ # Core types: Message, Role, ContentMeta, ILLMProvider
│   ├── Nao.Agents/ # Agent framework (ETCLOVG architecture)
│   │   ├── Shared/ # Cross-layer types (RetryPolicy)
│   │   ├── Core/ # IAgent, AgentId, AgentState, Tool (verify/revert), AgentAction
│   │   ├── Prompts/ # Prompt, PromptExample, OutputFormat
│   │   ├── Messaging/ # AgentMessage for inter-agent communication
│   │   ├── Logging/ # LogLevel, LogEntry, AgentLogger
│   │   ├── Environment/ # [E] ResourceLimits, SandboxConfig, ExecutionContext
│   │   ├── ToolProtocol/ # [T] ToolSchema, IToolProtocol, ToolRouter, ExecutionJournal
│   │   └── Memory/ # [C] ConversationWindow, MemoryStore, SemanticMemory,
│   │       ContextCompaction
│   ├── Lifecycle/ # [L] AgentLifecycle, LifecyclePipeline
│   ├── Orchestration/ # [L] Router, Pipeline, AgentGroup, Orchestrator
│   ├── Observability/ # [O] Trace, Metrics, Resilience (CircuitBreaker)
│   ├── Verification/ # [V] Verification, Regression
│   ├── Governance/ # [G] Permission, Constitution, AuditLog, PolicyEngine
│   └── Harness/ # EtclovgHarness (integrates all layers)
├── Nao.Eval/ # Evaluation framework: test cases, evaluators, LLM judge
├── Nao.Loader/ # Workspace loader: JSON defs, multi-mode execution, plugins
└── Nao.Providers/ # LLM provider implementations

```

```

├── Nao.Runtime.Orleans/ # Distributed runtime (grains, workspaces, groups)
│   ├── Workspace/ # WorkspaceRegistry (multi-tenant workspace isolation)
│   ├── Grains/ # SessionGrain, SessionDirectory, GroupDirectory
│   └── Nao.Assistant/ # Avalonia.FuncUI desktop chat app (embedded server + UI)
├── Domain/ # Contracts, AppSettings (theme/language persistence)
├── Server/ # Embedded ASP.NET Core + Orleans host, WS streaming
├── Client/ # NaoClient WebSocket client
├── Components/ # Theme, Localization, reusable FuncUI controls
├── Views/ # Shell, SessionView, SettingsView, BuilderView
├── tests/
├── Nao.Core.Tests/
├── Nao.Agents.Tests/ # Unit tests for all ETCLOVG layers
├── Nao.Eval.Tests/
├── Nao.Loader.Tests/
├── Nao.Providers.Tests/
├── Nao.Runtime.Orleans.Tests/
├── Nao.Assistant.Tests/
└── Nao.E2E.Tests/ # End-to-end: orchestration + full ETCLOVG harness demos

```

Prerequisites

- .NET 10.0+
- Paket (installed as a local tool)

Getting Started

```
# Restore tools
dotnet tool restore
```

```
# Install dependencies
dotnet paket install
```

```
# Build
dotnet build
```

```
# Run tests
dotnet test
```

Architecture

ETCLOVG Harness

The EtclovgHarness integrates all seven layers into a unified execution pipeline. Every agent execution flows through:

```

G: Governance (permissions + policy pre-check)
→ V: Verification (readiness gates)
→ L: Lifecycle (initialize + start)
→ O: Observability (trace spans + metrics)
→ E: Execution (sandboxed agent.RunAsync)
→ G: Constitution (output validation)
→ L: Lifecycle (complete)
→ V: Verification (trace store + regression + judge)
→ G: Audit (record)

```

```

let config =
{ EtclovgConfig.Default with
  Execution = SandboxConfig.Restricted (ResourceLimits.Constrained 60 50 100000)
  ToolProtocol = Some (ToolProtocol.fromTools myTools)
  Tracer = Some (Tracer.inMemory ())
  Metrics = Some (MetricsCollector.inMemory ())
  Constitution = Some (Constitution.empty "safety" |> Constitution.addRule
    Constitution.noPrivateDataRule)
  Permissions = Some (PermissionModel.Permissive agentId)

```

```
PolicyEngine = Some (PolicyEngine.create [ PolicyEngine.costBudgetPolicy 10.0m ])
ReadinessChecks = [ myReadinessCheck ]
TraceStore = Some traceStore
AuditLog = Some (AuditLog.inMemory ())
Lifecycle = [ myHook ] }
```

```
let! result = EtclovgHarness.runAsync config agent "What is the stock price?"
// result.Success, result.Response, result.Metrics, result.Trace,
result.HarnessError, ...
```

Structured errors via HarnessError DU:

```
match result.HarnessError with
| Some HarnessError.PermissionDenied -> ...
| Some (HarnessError.PolicyBlocked violations) -> ...
| Some (HarnessError.NotReady reasons) -> ...
| Some (HarnessError.ResourceLimitExceeded limit) -> ...
| Some (HarnessError.ConstitutionViolation ruleIds) -> ...
| None -> // success
```

Agent Model

Every agent implements IAgent:

```
type IAgent =
abstract member Id: AgentId
abstract member RunAsync: string -> Task<string>
abstract member HandleMessageAsync: AgentMessage -> Task<AgentMessage option>
abstract member State: AgentState
```

Agents can invoke tools, delegate to sub-agents, or respond directly:

```
type AgentAction =
| Respond of string
| InvokeTool of toolName: string * input: string
| DelegateToAgent of agentName: string * input: string
| Think of string
```

Orchestration Patterns

Router — A central agent decides which specialist handles the request:

```
let router = Router.create [ weatherAgent; mathAgent ] (ByPrompt orchestrator)
let result = Router.routeAsync "What's the weather?" router
```

Routing strategies: ByName, ByPrompt (LLM-decided), RoundRobin, Custom.

Pipeline — Sequential processing through multiple agents:

```
let pipeline = Pipeline.create [ fetcher; summarizer; formatter ]
let result = Pipeline.runAsync input pipeline
```

AgentGroup — Collaborative multi-agent conversation with termination conditions:

```
let group = AgentGroup.create [ analyst; critic ] (MaxRounds 5)
let history = AgentGroup.runAsync "Analyze this data" group
```

Custom Orchestrators

The Orchestrator uses an abstract base class (OrchestratorBase) with virtual members that can be overridden via inheritance. This solves the problem that function-valued fields (like action parsers) cannot be expressed in JSON configuration:

```
type MyOrchestrator(config: OrchestratorConfig) =
inherit OrchestratorBase(config)
```

```
override _.TryParseAction(content) =
```

```
// Custom parsing logic for your LLM's output format
if content.Contains("<tool>") then
Some (InvokeTool ("myTool", content))
else
None

override _.BuildSystemPrompt() =
"You are a domain-specific assistant. Use XML tags to invoke tools."

override _.OnToolResult(toolName, input, result) =
printfn "Tool %s returned: %s" toolName result

override _.OnRoundComplete(round, content) =
printfn "Round %d complete" round
```

Register a custom factory via DI to have the runtime use your subclass:

```
type MyOrchestratorFactory() =
interface IOrchestratorFactory with
member _.Create(config) = MyOrchestrator(config) :> IAgent
```

Available virtual members on OrchestratorBase:

Member	Purpose
BuildSystemPrompt()	Customize system prompt generation
TryParseAction(content)	Parse LLM output into tool/agent actions
OnToolResult(name, input, result)	Hook after tool execution
OnRoundComplete(round, content)	Hook after each reasoning round

Memory Management

Conversation Windowing — Prevent token overflow:

```
type WindowStrategy =
| LastN of int // Keep last N messages
| TokenBudget of maxTokens: int // Fit within token limit
| SummarizeAfter of threshold: int // Summarize old messages
```

Summarization — LLM-powered condensation of older messages:

```
let config = SummarizationConfig.Default provider
let trimmed = Summarizer.applyAsync config conversation
```

Key-Value Memory — Structured fact storage per agent:

```
let store = InMemoryStore() :> IMemoryStore
store.SaveAsync agentId { Key = "user-name"; Value = "Alice"; ... }
store.RecallAsync agentId "user"
```

Semantic Memory — Embedding-based similarity retrieval:

```
let memory = InMemorySemanticMemory(embeddingProvider) :> ISemanticMemory
memory.StoreAsync agentId "fact-1" "The capital of France is Paris"
memory.RetrieveAsync agentId "What's the French capital?" topK=3
```

Tool Protocol (T)

MCP-inspired tool discovery with middleware:

```
// Create protocol with rate limiting
let protocol =
ToolProtocol.fromTools myTools
|> ToolProtocol.withMiddleware (ToolProtocol.rateLimitMiddleware 100)

// Discovery
let! schemas = protocol.ListTools()
let! available = protocol.IsAvailable "get_weather"
```

```
// Invocation with structured result
let! result = protocol.InvokeAsync "get_weather" "London"
// result.Success, result.Output, result.DurationMs, result.Error
```

Content Metadata

Tools and agents declare their output type via ContentMeta:

```
let meta = ContentMeta.Json
let custom = ContentMeta.WithMeta "image/png" [ "width", "1024"; "height", "768" ]
```

Tool Verify & Revert

Tools can optionally verify correctness and undo side-effects:

```
let tool =
{ Tool.Create("deploy", "Deploy to staging", fun input -> task { ... }) with
Verify = Some (fun input output -> task {
// Check the deployment was successful
return Ok ()
})
Revert = Some (fun ctx -> task {
// Rollback the deployment
return Ok ()
}) }
```

Execution Journal

Immutable audit log of all tool executions; enables bulk revert:

```
let journal = InMemoryExecutionJournal() :> IExecutionJournal
```

```
// Revert all revertible operations
let! failures = ExecutionJournal.revertAllAsync journal tools
```

Governance (G)

Permission Model — Control which tools/capabilities agents can access:

```
let perms =
PermissionModel.Permissive agentId
|> PermissionModel.grant "tool:search" PermissionLevel.Allow
|> PermissionModel.grant "tool:delete" PermissionLevel.Deny
```

Constitution — Rules that agent outputs must satisfy:

```
let constitution =
Constitution.empty "safety"
|> Constitution.addRule Constitution.noPrivateDataRule
|> Constitution.addRule Constitution.noHarmRule
let result = Constitution.check constitution agentOutput
// result.Passed, result.Violations, hasHardViolations
```

Policy Engine — Budget enforcement, rate limiting, content policies:

```
let engine = PolicyEngine.create [
PolicyEngine.costBudgetPolicy 5.0m
PolicyEngine.rateLimitPolicy "tool_call" 60
]
let result = engine.Evaluate(PolicyContext.FromExecutionContext agentId "execute"
input ctx)
```

Observability (O)

Distributed Tracing — OpenTelemetry-style spans:

```
let tracer = Tracer.inMemory ()
let root = tracer.StartTrace "user-request"
```

```
let child = tracer.StartSpan root "tool.invoke"
tracer.EndSpan child SpanStatus.Ok
```

Metrics — Token usage, cost tracking, latency percentiles:

```
let metrics = MetricsCollector.inMemory ()
metrics.RecordLlmCall inputTokens outputTokens latencyMs
let cost = metrics.EstimateCost MetricsCollector.gpt4o
let summary = metrics.GetMetrics() // TotalLlmCalls, AvgLatencyMs, P95, ...
```

Resilience — Retry with backoff, circuit breakers, fallbacks:

```
let config = { ResilienceConfig.Default with
  RetryPolicy = RetryPolicy.ExponentialBackoff (3, 1000, 30000)
  Fallback = FallbackStrategy.DefaultValue "cached result" }
let! result = Resilience.executeAsync config (Some circuitBreaker) myFunc input
```

Verification (V)

Readiness Gates — Pre-flight checks before execution:

```
let! readiness = Verification.checkReadiness [ toolCheck; budgetCheck ] agentId
input
match readiness with
| ReadinessResult.Ready -> // proceed
| ReadinessResult.NotReady reasons -> // block
```

Execution Traces — Full step-by-step history for analysis:

```
let trace =
  Verification.startTrace agentId input
|> Verification.addStep (TraceAction.LlmCall "gpt-4o") input output 150L
|> Verification.addStep (TraceAction.ToolInvocation "search") query result 25L
|> Verification.complete finalOutput
```

Regression Detection — Compare against baselines:

```
let regression = Regression.detect baselineTrace currentTrace
// regression.IsRegression, regression.Regressions (latency, quality, cost)
```

Evaluation (Nao.Eval)

Run agents against datasets with multiple evaluators:

```
let dataset = { Name = "math"; Cases = [ EvalCase.create "1" "2+2" (Some "4") ] }
let! report = EvalRunner.runDatasetAsync evaluator agent dataset
EvalRunnerConfig.Default
// report.PassRate, report.AverageScore, report.TagBreakdown
```

Built-in evaluators: ExactMatch, Contains, Regex, LlmJudge, Composite.

Orleans Runtime

Agents run as Orleans grains for distributed, persistent execution:

- SessionGrain — Full ETCLOVG-integrated session with multi-conversation support
- SessionDirectoryGrain — Tracks all sessions per user
- GroupDirectoryGrain — Organizational multi-tenancy with member/session management
- WorkspaceRegistry — Multiple isolated workspaces within a single silo

```
// Register multiple workspaces in the silo
let registry = WorkspaceRegistry.fromWorkspaces [
  WorkspaceId.create "team-a", loadedDefsA
  WorkspaceId.create "team-b", loadedDefsB
]
```

```
// Sessions resolve agents/tools from their workspace
let options = { AgentName = "assistant"; WorkspaceKey = "team-a"; GroupId = Some
```

```
"org-1"; ToolNames = [] }  
sessionGrain.StartAsync(options)
```

```
// Switch workspace at runtime without losing conversation  
sessionGrain.SwitchWorkspaceAsync("team-b")
```

Group Directory

Organizational isolation — groups manage members, sessions, and default workspaces:

```
let groupGrain = clusterClient.GetGrain<IGroupDirectoryGrain>("org-1")  
groupGrain.InitAsync("Engineering", "team-a")  
groupGrain.AddMemberAsync("user-123", "admin")  
groupGrain.RegisterSessionAsync(entry)  
let! sessions = groupGrain.ListUserSessionsAsync("user-123")
```

Structured Prompts

```
let prompt =  
{ Prompt.Empty with  
  Role = "You are a financial analyst."  
  Objective = "Analyze quarterly earnings reports."  
  Constraints = [ "Use only provided data"; "Be concise" ]  
  Examples = [ { Input = "Q1 revenue?"; Output = "$2.3B"; Explanation = None } ]  
  OutputFormat = Json (Some ""{"summary": "...", "trend": "..."}"" ) }
```

Desktop Assistant

Nao.Assistant is a cross-platform desktop chat client built with Avalonia and Avalonia.FuncUI, styled with Semi.Avalonia. It hosts an **embedded ASP.NET Core + Orleans server** in-process, so a single executable provides both the runtime and the UI.

```
dotnet run --project src/Nao.Assistant
```

Highlights:

- **Live execution trace** — As a turn runs, the assistant streams what it is doing (reasoning, tool calls, sub-agent steps) over a WebSocket and renders the process *above* the final answer in real time, instead of hiding it behind a "details" toggle.
 - **Theme switching** — Dark and light themes are selectable from Settings, applied instantly via centralized design tokens, and persisted across launches.
 - **Localizable UI** — UI strings flow through a central Localization table (English today), with a language selector wired into Settings for future translations.
 - **Persisted preferences** — Theme, language, provider, orchestrator, and workspace settings are stored in AppSettings and reloaded on startup.
- LLM turns can run well beyond Orleans' default 30s grain-response timeout, so the embedded host raises the silo/client ResponseTimeout accordingly.

Package Management

This project uses Paket for dependency management. To add a package:

1. Edit paket.dependencies to add the source package
2. Add the package name to the relevant project's paket.references
3. Run `dotnet paket install`

Git Hooks

A pre-commit hook ensures all tests pass before commits are accepted. It runs `dotnet test` automatically.

Coding Conventions

File Organization

- **One type per file** — Each type, interface, or discriminated union gets its own file
- **File names match the primary type** — e.g. `AgentState` lives in `AgentState.fs`
- **Compile order matters** — Files in `.fsproj` are listed in dependency order (dependencies first)

Naming

- **Types**: PascalCase (`CompletionResult`, `AgentGroup`)
- **Modules**: PascalCase, matching the type they operate on (module `ConversationWindow`)
- **Functions**: camelCase (`applyLastN`, `routeAsync`)
- **DU cases**: PascalCase (`LastN`, `TokenBudget`, `ByPrompt`)
- **Interfaces**: prefix with `I` (`ILlmProvider`, `IAgent`, `IMemoryStore`)

F# Style

- Prefer discriminated unions over class hierarchies
- Prefer immutable records for data types
- Use `option` instead of `null`
- Use `Task<T>` for async operations (interop-friendly)
- Keep modules alongside their corresponding type for helper functions
- Use XML doc comments (`///`) for public API types and members

Project Structure

- Source projects go under `src/`
- Test projects go under `tests/`
- Each source project has a matching `<ProjectName>.Tests` project
- Test projects use MSTest framework
- Dependencies between source projects use `<ProjectReference>`

Testing

- Test project names: `<ProjectName>.Tests`
- Test framework: MSTest
- One test file per feature or module being tested
- Test methods should be descriptive: `OrchestratorRoutesToWeatherAgent`

License

MIT